

DocAI 完整資料庫結構文檔

Complete Database Schema Documentation

生成日期: 2026-02-10 系統版本: DocAI RAG Application v1.0.0 文檔目的: 容器化部署時的資料庫初始化參考

目錄 / Table of Contents

- [1. SQLite - Skill Metadata Database](#)
- [2. SQLite - File Metadata Database](#)
- [3. MongoDB - Chat History Database](#)
- [4. Redis - Cache Layer](#)
- [5. FAISS - Vector Store](#)

1. SQLite - Skill Metadata Database

檔案位置: /app/data/skill_metadata.db **Provider:** app/Providers/skill_metadata_provider/client.py **驅動:**aiosqlite (async SQLite)

1.1 skill_heads 表 (技能頭定義)

-- 技能分組頭表 - 定義技能的頂層結構
-- Source: client.py lines 142-157

```
CREATE TABLE IF NOT EXISTS skill_heads (  
    head_id TEXT PRIMARY KEY,          -- 唯一識別碼 (format: head_YYYYMMDD_HHMM)  
    skill_name TEXT NOT NULL,          -- 技能顯示名稱  
    skill_description TEXT,            -- 技能描述  
    skill_category TEXT DEFAULT 'general', -- 技能分類  
    icon TEXT DEFAULT ' ',             -- 顯示圖示  
    color TEXT DEFAULT '#4a90d9',     -- 主題色彩  
    is_active INTEGER DEFAULT 1,       -- 是否啟用 (1=是, 0=否)  
    display_order INTEGER DEFAULT 0,   -- 顯示排序  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
-- 索引  
CREATE INDEX IF NOT EXISTS idx_skill_heads_name  
    ON skill_heads(skill_name);  
CREATE INDEX IF NOT EXISTS idx_skill_heads_category  
    ON skill_heads(skill_category);  
CREATE INDEX IF NOT EXISTS idx_skill_heads_active  
    ON skill_heads(is_active);
```

1.2 skill_metadata 表 (技能文件元資料)

-- 技能文件元資料表 - 存儲每個上傳文件的處理資訊
-- Source: client.py lines 159-186

```
CREATE TABLE IF NOT EXISTS skill_metadata (  
    skill_id TEXT PRIMARY KEY,          -- 唯一識別碼 (format: skill_YYYYMMDD_HHI  
    head_id TEXT,                      -- 關聯的 skill_head (外鍵)  
    skill_name TEXT NOT NULL,          -- 技能名稱  
    skill_description TEXT,            -- 技能描述  
    skill_category TEXT DEFAULT 'general', -- 技能分類  
    source_type TEXT DEFAULT 'pdf',    -- 來源類型 (pdf/docx/pptx/txt/md)  
    source_name TEXT,                  -- 來源檔案名稱 (不含副檔名)  
    total_chunks INTEGER DEFAULT 0,    -- 總 chunk 數量  
    embedding_model TEXT,               -- 使用的嵌入模型  
    embedding_dimension INTEGER,        -- 嵌入向量維度  
    chunk_size INTEGER DEFAULT 1000,   -- chunk 大小  
    chunk_overlap INTEGER DEFAULT 200, -- chunk 重疊  
    parent_skill_id TEXT DEFAULT 'root', -- 父技能 ID (用於即時附件)  
    faiss_index_path TEXT,              -- FAISS 索引檔案路徑  
    status TEXT DEFAULT 'active',       -- 狀態 (active/inactive/processing)  
    metadata TEXT,                      -- JSON 格式的額外元資料  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
    FOREIGN KEY (head_id) REFERENCES skill_heads(head_id)  
);
```

-- 索引

```
CREATE INDEX IF NOT EXISTS idx_skill_metadata_name  
    ON skill_metadata(skill_name);  
CREATE INDEX IF NOT EXISTS idx_skill_metadata_category  
    ON skill_metadata(skill_category);  
CREATE INDEX IF NOT EXISTS idx_skill_metadata_head_id  
    ON skill_metadata(head_id);  
CREATE INDEX IF NOT EXISTS idx_skill_metadata_parent  
    ON skill_metadata(parent_skill_id);  
CREATE INDEX IF NOT EXISTS idx_skill_metadata_status  
    ON skill_metadata(status);
```

1.3 skill_chunk_metadata 表 (Chunk 詳細元資料)

-- Chunk 層級的詳細元資料
-- Source: client.py lines 188-211

```
CREATE TABLE IF NOT EXISTS skill_chunk_metadata (  
    chunk_id TEXT PRIMARY KEY,          -- 唯一識別碼
```

```

    skill_id TEXT NOT NULL,                -- 關聯的 skill_metadata
    chunk_index INTEGER NOT NULL,          -- chunk 在文件中的順序
    content_hash TEXT,                    -- 內容雜湊 (用於去重)
    char_count INTEGER,                   -- 字元數
    token_count INTEGER,                  -- token 數 (估計)
    page_number INTEGER,                  -- 原始頁碼
    section_title TEXT,                   -- 段落標題
    metadata TEXT,                        -- JSON 格式的額外元資料
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (skill_id) REFERENCES skill_metadata(skill_id)
);

-- 索引
CREATE INDEX IF NOT EXISTS idx_chunk_skill_id
    ON skill_chunk_metadata(skill_id);
CREATE INDEX IF NOT EXISTS idx_chunk_content_hash
    ON skill_chunk_metadata(content_hash);
CREATE INDEX IF NOT EXISTS idx_chunk_page
    ON skill_chunk_metadata(page_number);

```

1.4 skill_document_mapping 表 (文件映射)

```

-- 技能與文件的多對多映射關係
-- Source: client.py lines 213-230

CREATE TABLE IF NOT EXISTS skill_document_mapping (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    skill_id TEXT NOT NULL,                -- skill_metadata.skill_id
    document_id TEXT NOT NULL,             -- 外部文件識別碼
    document_name TEXT,                   -- 文件名稱
    document_path TEXT,                   -- 文件路徑
    chunk_start INTEGER,                  -- 起始 chunk 索引
    chunk_end INTEGER,                    -- 結束 chunk 索引
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (skill_id) REFERENCES skill_metadata(skill_id),
    UNIQUE(skill_id, document_id)
);

-- 索引
CREATE INDEX IF NOT EXISTS idx_mapping_skill_id
    ON skill_document_mapping(skill_id);
CREATE INDEX IF NOT EXISTS idx_mapping_document_id
    ON skill_document_mapping(document_id);

```

1.5 skill_overviews 表 (技能概述)

```

-- 技能概述摘要 (用於快速顯示)

```

-- Source: client.py lines 232-252

```
CREATE TABLE IF NOT EXISTS skill_overviews (  
    overview_id TEXT PRIMARY KEY,  
    skill_id TEXT NOT NULL,                -- 關聯的 skill_metadata  
    summary TEXT,                          -- 自動生成的摘要  
    key_topics TEXT,                       -- JSON 陣列的關鍵主題  
    difficulty_level TEXT,                 -- 難度等級 (beginner/intermediate/advance)  
    estimated_read_time INTEGER,           -- 預估閱讀時間 (分鐘)  
    language TEXT DEFAULT 'zh-TW',         -- 主要語言  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
    FOREIGN KEY (skill_id) REFERENCES skill_metadata(skill_id)  
);  
  
-- 索引  
CREATE INDEX IF NOT EXISTS idx_overview_skill_id  
    ON skill_overviews(skill_id);
```

1.6 processing_jobs 表 (處理任務)

-- 非同步處理任務追蹤

-- Source: client.py lines 254-280

```
CREATE TABLE IF NOT EXISTS processing_jobs (  
    job_id TEXT PRIMARY KEY,               -- 任務唯一識別碼  
    skill_id TEXT,                         -- 關聯的 skill_metadata  
    job_type TEXT NOT NULL,                 -- 任務類型 (ingest/rebuild/delete)  
    status TEXT DEFAULT 'pending',         -- 狀態 (pending/processing/completed/  
    progress INTEGER DEFAULT 0,            -- 進度百分比 (0-100)  
    total_steps INTEGER,                   -- 總步驟數  
    current_step INTEGER DEFAULT 0,        -- 當前步驟  
    error_message TEXT,                    -- 錯誤訊息  
    started_at TIMESTAMP,                  -- 開始時間  
    completed_at TIMESTAMP,                -- 完成時間  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
    FOREIGN KEY (skill_id) REFERENCES skill_metadata(skill_id)  
);  
  
-- 索引  
CREATE INDEX IF NOT EXISTS idx_job_skill_id  
    ON processing_jobs(skill_id);  
CREATE INDEX IF NOT EXISTS idx_job_status  
    ON processing_jobs(status);  
CREATE INDEX IF NOT EXISTS idx_job_type  
    ON processing_jobs(job_type);
```

2. SQLite - File Metadata Database

檔案位置: /app/data/docai.db **Provider:** app/Providers/
file_metadata_provider/client.py **驅動:** aiosqlite (async SQLite)

2.1 file_metadata 表 (檔案元資料)

```
-- File-Based RAG 系統的檔案元資料
-- Source: file_metadata_provider/client.py lines 96-115

CREATE TABLE IF NOT EXISTS file_metadata (
    file_id TEXT PRIMARY KEY,          -- 唯一識別碼 (UUID)
    file_name TEXT NOT NULL,           -- 原始檔案名稱
    file_path TEXT NOT NULL,           -- 儲存路徑
    file_type TEXT NOT NULL,           -- 檔案類型 (pdf/docx/pptx/txt/md)
    file_size INTEGER,                 -- 檔案大小 (bytes)
    file_hash TEXT,                    -- SHA-256 雜湊
    total_pages INTEGER,               -- 總頁數
    total_chunks INTEGER DEFAULT 0,    -- 總 chunk 數
    processing_status TEXT DEFAULT 'pending', -- 處理狀態
    faiss_index_path TEXT,              -- FAISS 索引路徑
    embedding_model TEXT,               -- 嵌入模型名稱
    metadata TEXT,                      -- JSON 格式額外資料
    uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    processed_at TIMESTAMP,             -- 處理完成時間
    last_accessed_at TIMESTAMP         -- 最後存取時間
);

-- 索引
CREATE INDEX IF NOT EXISTS idx_file_name
    ON file_metadata(file_name);
CREATE INDEX IF NOT EXISTS idx_file_type
    ON file_metadata(file_type);
CREATE INDEX IF NOT EXISTS idx_file_hash
    ON file_metadata(file_hash);
CREATE INDEX IF NOT EXISTS idx_processing_status
    ON file_metadata(processing_status);
```

2.2 chunks_metadata 表 (Chunk 元資料)

```
-- File-Based 系統的 chunk 元資料
-- Source: file_metadata_provider/client.py lines 117-136

CREATE TABLE IF NOT EXISTS chunks_metadata (
    chunk_id TEXT PRIMARY KEY,          -- 唯一識別碼
    file_id TEXT NOT NULL,              -- 關聯的 file_metadata
```

```

    chunk_index INTEGER NOT NULL,          -- chunk 順序索引
    content TEXT,                          -- chunk 文字內容
    content_hash TEXT,                    -- 內容雜湊
    char_count INTEGER,                   -- 字元數
    page_number INTEGER,                  -- 原始頁碼
    embedding_vector BLOB,                 -- 嵌入向量 (可選快取)
    metadata TEXT,                        -- JSON 格式額外資料
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (file_id) REFERENCES file_metadata(file_id)
);

-- 索引
CREATE INDEX IF NOT EXISTS idx_chunk_file_id
    ON chunks_metadata(file_id);
CREATE INDEX IF NOT EXISTS idx_chunk_index
    ON chunks_metadata(chunk_index);
CREATE INDEX IF NOT EXISTS idx_chunk_hash
    ON chunks_metadata(content_hash);

```

3. MongoDB - Chat History Database

連線 **URI**: mongodb://mongodb:27017 (容器內) 或 mongodb://localhost:27017 (本機)
 資料庫名稱: docai **Provider**: app/Providers/chat_history_provider/client.py 驅動: motor (async MongoDB)

3.1 chat_sessions 集合 (聊天會話)

```

// MongoDB 集合結構
// Source: chat_history_provider/client.py lines 84-86

// 集合名稱: chat_sessions
// 文件結構:
{
    "_id": ObjectId("..."),          // MongoDB 自動生成
    "session_id": "sess_20260210_uuid", // 會話唯一識別碼
    "user_id": "user_123",             // 用戶識別碼 (可選)
    "skill_id": "skill_xxx",           // 關聯的技能 ID (可選)
    "file_id": "file_xxx",             // 關聯的檔案 ID (可選)
    "mode": "skill",                   // 模式 (skill/file)
    "messages": [                      // 訊息陣列
        {
            "role": "user",             // 角色 (user/assistant/system)
            "content": "用戶問題...",
            "timestamp": ISODate("2026-02-10T10:30:00Z"),
            "metadata": {                // 可選的額外資料

```

```

        "source_chunks": [...],
        "relevance_scores": [...]
    }
},
{
    "role": "assistant",
    "content": "AI 回答...",
    "timestamp": ISODate("2026-02-10T10:30:05Z"),
    "metadata": {
        "model": "gpt-oss:20b",
        "tokens_used": 512,
        "sources": [...]
    }
}
],
"context": {
    "system_prompt": "...",
    "temperature": 0.0,
    "max_tokens": 2048
},
"created_at": ISODate("2026-02-10T10:00:00Z"),
"updated_at": ISODate("2026-02-10T10:30:05Z"),
"is_active": true,
"metadata": {}
}

```

// 索引建立

// Source: chat_history_provider/client.py lines 84-86

```

db.chat_sessions.createIndex({ "session_id": 1 }, { unique: true });
db.chat_sessions.createIndex({ "user_id": 1 });
db.chat_sessions.createIndex({ "created_at": -1 });
db.chat_sessions.createIndex({ "skill_id": 1 });
db.chat_sessions.createIndex({ "file_id": 1 });
db.chat_sessions.createIndex({ "is_active": 1 });

```

// TTL 索引 (可選 - 自動刪除過期會話)

```

db.chat_sessions.createIndex(
    { "created_at": 1 },
    { expireAfterSeconds: 2592000 } // 30 天後自動刪除
);

```

3.2 MongoDB 初始化腳本

// 初始化腳本 (可放入 docker-entrypoint-initdb.d/)

// File: init-mongo.js

// 切換到 docai 資料庫

use docai;

// 建立集合 (帶驗證規則)

```

db.createCollection("chat_sessions", {

```

```

validator: {
  $jsonSchema: {
    bsonType: "object",
    required: ["session_id", "messages", "created_at"],
    properties: {
      session_id: {
        bsonType: "string",
        description: "唯一會話識別碼"
      },
      messages: {
        bsonType: "array",
        description: "訊息陣列",
        items: {
          bsonType: "object",
          required: ["role", "content", "timestamp"],
          properties: {
            role: {
              enum: ["user", "assistant", "system"],
              description: "訊息角色"
            },
            content: {
              bsonType: "string",
              description: "訊息內容"
            },
            timestamp: {
              bsonType: "date",
              description: "訊息時間戳"
            }
          }
        }
      },
      created_at: {
        bsonType: "date",
        description: "建立時間"
      }
    }
  }
}
});

// 建立索引
db.chat_sessions.createIndex({ "session_id": 1 }, { unique: true });
db.chat_sessions.createIndex({ "user_id": 1 });
db.chat_sessions.createIndex({ "created_at": -1 });

print("MongoDB initialization completed for DocAI");

```

4. Redis - Cache Layer

連線 URI: redis://redis:6379/0 (容器內) 或 redis://localhost:6379/0 (本機)

Provider: app/Providers/cache_provider/client.py **驅動:** redis.asyncio (async Redis)

4.1 快取鍵值模式 (Key Patterns)

```
# Redis 快取鍵值設計
# Source: cache_provider/client.py

# =====
# 1. 嵌入向量快取 (Embedding Cache)
# =====
# 用途: 避免重複計算相同文字的嵌入向量
# TTL: 3600 秒 (1 小時)

KEY_PATTERN = "emb:{model}:{content_hash}"

# 範例:
"emb:bge-m3:a1b2c3d4e5f6..." # BGE-M3 模型的嵌入快取
"emb:minilm:x9y8z7w6v5u4..." # MiniLM 模型的嵌入快取

# 值格式: JSON 序列化的向量陣列
# {"embedding": [0.123, -0.456, 0.789, ...], "dimension": 1024}

# =====
# 2. 查詢擴展快取 (Query Expansion Cache)
# =====
# 用途: 快取 LLM 產生的查詢擴展結果
# TTL: 1800 秒 (30 分鐘)

KEY_PATTERN = "qexp:{query_hash}"

# 範例:
"qexp:abc123def456..."

# 值格式: JSON 陣列的擴展查詢
# ["original query", "expanded query 1", "expanded query 2"]

# =====
# 3. 搜尋結果快取 (Search Results Cache)
# =====
# 用途: 快取相同查詢的檢索結果
# TTL: 600 秒 (10 分鐘)

KEY_PATTERN = "search:{skill_id}:{query_hash}"

# 範例:
"search:skill_20260210_abc:xyz789..."
```

```

# 值格式: JSON 序列化的檢索結果
# {"chunks": [...], "scores": [...], "metadata": {...}}

# =====
# 4. 檔案處理狀態快取 (File Processing Status)
# =====
# 用途: 追蹤檔案處理進度
# TTL: 7200 秒 (2 小時)

KEY_PATTERN = "file:{file_id}:status"

# 範例:
"file:file_20260210_abc:status"

# 值格式: JSON 狀態物件
# {"status": "processing", "progress": 45, "step": "embedding"}

# =====
# 5. 會話上下文快取 (Session Context Cache)
# =====
# 用途: 快速存取活躍會話的上下文
# TTL: 3600 秒 (1 小時)

KEY_PATTERN = "session:{session_id}:context"

# 範例:
"session:sess_20260210_abc:context"

# 值格式: JSON 序列化的會話上下文
# {"messages": [...], "skill_id": "...", "settings": {...}}

# =====
# 6. 技能索引元資料快取 (Skill Index Metadata)
# =====
# 用途: 快取技能索引的元資料以加速載入
# TTL: 86400 秒 (24 小時)

KEY_PATTERN = "skill:{skill_id}:meta"

# 範例:
"skill:skill_20260210_abc:meta"

# 值格式: JSON 序列化的索引元資料
# {"total_chunks": 500, "dimension": 1024, "last_updated": "..."}

```

4.2 Redis 初始化與配置

```

# Redis 連線與初始化
# Source: cache_provider/client.py lines 45-68

import redis.asyncio as redis
from typing import Optional
import json

class CacheProvider:
    def __init__(self):
        self.redis_url = os.getenv("REDIS_URL", "redis://localhost:6379/0")
        self.default_ttl = int(os.getenv("CACHE_TTL", 3600))
        self._client: Optional[redis.Redis] = None

    async def connect(self):
        """建立 Redis 連線"""
        self._client = redis.from_url(
            self.redis_url,
            encoding="utf-8",
            decode_responses=True
        )
        # 測試連線
        await self._client.ping()

    async def get(self, key: str) -> Optional[dict]:
        """取得快取值"""
        value = await self._client.get(key)
        return json.loads(value) if value else None

    async def set(self, key: str, value: dict, ttl: int = None):
        """設定快取值"""
        ttl = ttl or self.default_ttl
        await self._client.setex(key, ttl, json.dumps(value))

    async def delete(self, key: str):
        """刪除快取"""
        await self._client.delete(key)

    async def clear_pattern(self, pattern: str):
        """清除符合模式的所有快取"""
        keys = await self._client.keys(pattern)
        if keys:
            await self._client.delete(*keys)

```

5. FAISS - Vector Store

儲存位置: /app/data/faiss_indices/ **Provider:** app/Providers/
 vector_store_provider/client.py **驅動:** faiss-cpu +
 langchain_community.vectorstores

5.1 FAISS 索引結構

```
# FAISS 向量索引建立
# Source: vector_store_provider/client.py lines 250-290

import faiss
from langchain_community.vectorstores import FAISS
from langchain_community.docstore.in_memory import InMemoryDocstore

# =====
# 索引類型: IndexFlatL2 (L2 距離精確搜尋)
# =====

# Skill-Based 系統使用 BGE-M3 (1024 維)
EMBEDDING_DIMENSION_SKILLS = 1024

# File-Based 系統使用 MiniLM (384 維)
EMBEDDING_DIMENSION_FILES = 384

def create_faiss_index(dimension: int) -> faiss.IndexFlatL2:
    """建立 FAISS 向量索引"""
    # IndexFlatL2: 精確 L2 距離搜尋 (無壓縮)
    index = faiss.IndexFlatL2(dimension)
    return index

# =====
# 目錄結構
# =====

# Skill 索引目錄
# /app/data/faiss_indices/skills/
# └─ skill_20260210_abc123/
#   └─ index.faiss          # FAISS 二進位索引檔
#   └─ index.pkl            # 文件存儲 (pickle 序列化)
# └─ skill_20260210_def456/
#   └─ index.faiss
#   └─ index.pkl
# └─ ...

# File 索引目錄
# /app/data/faiss_indices/files/
# └─ file_20260210_xyz789/
#   └─ index.faiss
#   └─ index.pkl
# └─ ...

# =====
# LangChain FAISS 封裝
# =====

from langchain_community.vectorstores import FAISS
```

```

from langchain_huggingface import HuggingFaceEmbeddings

def create_skill_vector_store(
    texts: list[str],
    metadatas: list[dict],
    skill_id: str
) -> str:
    """建立技能向量存儲並返回索引路徑"""

    # 初始化嵌入模型 (BGE-M3)
    embeddings = HuggingFaceEmbeddings(
        model_name="BAAI/bge-m3",
        model_kwargs={"device": "cpu"},
        encode_kwargs={"normalize_embeddings": True}
    )

    # 建立 FAISS 向量存儲
    vector_store = FAISS.from_texts(
        texts=texts,
        embedding=embeddings,
        metadatas=metadatas
    )

    # 儲存索引
    save_path = f"/app/data/faiss_indices/skills/{skill_id}"
    vector_store.save_local(save_path)

    return save_path

def load_skill_vector_store(skill_id: str) -> FAISS:
    """載入技能向量存儲"""

    embeddings = HuggingFaceEmbeddings(
        model_name="BAAI/bge-m3",
        model_kwargs={"device": "cpu"},
        encode_kwargs={"normalize_embeddings": True}
    )

    load_path = f"/app/data/faiss_indices/skills/{skill_id}"
    vector_store = FAISS.load_local(
        load_path,
        embeddings,
        allow_dangerous_deserialization=True # 信任本地索引
    )

    return vector_store

```

5.2 FAISS 索引檔案格式

```

# =====
# index.faiss 檔案結構 (二進位格式)

```

```

# =====

# FAISS 原生二進位格式，包含：
# - 索引類型標識
# - 向量維度
# - 向量數量
# - 所有向量資料 (float32 陣列)
# - 索引特定的元資料

# 檔案大小估算：
# size ≈ num_vectors × dimension × 4 bytes (float32)
# 例：1000 chunks × 1024 dim × 4 = ~4 MB

# =====
# index.pkl 檔案結構 (Pickle 序列化)
# =====

# LangChain 文件存儲，包含：
{
    "docstore": InMemoryDocstore({
        "doc_id_0": Document(
            page_content="chunk 文字內容...",
            metadata={
                "skill_id": "skill_xxx",
                "chunk_index": 0,
                "page_number": 1,
                "source_file": "document.pdf",
                # ... 其他元資料
            }
        ),
        "doc_id_1": Document(...),
        # ...
    }),
    "index_to_docstore_id": {
        0: "doc_id_0",
        1: "doc_id_1",
        # FAISS 向量索引 → 文件 ID 映射
    }
}

```

5.3 FAISS 操作 API

```

# =====
# 向量存儲操作
# =====

class VectorStoreProvider:
    """向量存儲提供者"""

```

```

async def create_index(
    self,
    skill_id: str,
    texts: list[str],
    metadatas: list[dict],
    store_type: str = "skill" # "skill" or "file"
) -> str:
    """建立新索引"""

    # 選擇嵌入模型
    if store_type == "skill":
        model_name = "BAAI/bge-m3"
        dimension = 1024
        base_path = "/app/data/faiss_indices/skills"
    else:
        model_name = "sentence-transformers/all-MiniLM-L6-v2"
        dimension = 384
        base_path = "/app/data/faiss_indices/files"

    embeddings = HuggingFaceEmbeddings(model_name=model_name)

    # 建立 FAISS
    vector_store = FAISS.from_texts(texts, embeddings, metadatas)

    # 儲存
    save_path = f"{base_path}/{skill_id}"
    vector_store.save_local(save_path)

    return save_path

async def search(
    self,
    skill_id: str,
    query: str,
    top_k: int = 10,
    store_type: str = "skill"
) -> list[tuple]:
    """相似度搜尋"""

    vector_store = await self.load_index(skill_id, store_type)

    # 執行搜尋
    results = vector_store.similarity_search_with_score(
        query=query,
        k=top_k
    )

    return results # [(Document, score), ...]

async def delete_index(self, skill_id: str, store_type: str = "skill"):
    """刪除索引"""
    import shutil

```

```

        if store_type == "skill":
            path = f"/app/data/faiss_indices/skills/{skill_id}"
        else:
            path = f"/app/data/faiss_indices/files/{skill_id}"

        if os.path.exists(path):
            shutil.rmtree(path)

    async def merge_indices(
        self,
        skill_ids: list[str],
        target_skill_id: str,
        store_type: str = "skill"
    ) -> str:
        """合併多個索引為一個主索引"""

        # 載入第一個索引作為基礎
        base_store = await self.load_index(skill_ids[0], store_type)

        # 合併其餘索引
        for skill_id in skill_ids[1:]:
            other_store = await self.load_index(skill_id, store_type)
            base_store.merge_from(other_store)

        # 儲存合併後的索引
        save_path = await self.create_index(...)
        return save_path

```

6. 資料庫初始化順序

6.1 容器啟動時的初始化流程

docker-compose.yml 中的依賴關係確保正確順序

```

services:
  docai-app:
    depends_on:
      mongodb:
        condition: service_healthy    # 1. 等待 MongoDB 健康
      redis:
        condition: service_healthy    # 2. 等待 Redis 健康
    # 3. DocAI 應用啟動時自動初始化 SQLite 和 FAISS

```

6.2 應用程式初始化代碼

main.py 中的初始化邏輯

```

async def initialize_databases():

```



```
"""初始化所有資料庫連線和結構"""
```

```
# 1. SQLite - 自動建立表格
```

```
skill_metadata_provider = SkillMetadataProvider()
```

```
await skill_metadata_provider.initialize() # 建立 skill_metadata.db 的所有表
```

```
file_metadata_provider = FileMetadataProvider()
```

```
await file_metadata_provider.initialize() # 建立 docai.db 的所有表
```

```
# 2. MongoDB - 建立連線和索引
```

```
chat_provider = ChatHistoryProvider()
```

```
await chat_provider.connect()
```

```
await chat_provider.create_indexes() # 建立 MongoDB 索引
```

```
# 3. Redis - 建立連線
```

```
cache_provider = CacheProvider()
```

```
await cache_provider.connect()
```

```
# 4. FAISS - 確保目錄存在
```

```
os.makedirs("/app/data/faiss_indices/skills", exist_ok=True)
```

```
os.makedirs("/app/data/faiss_indices/files", exist_ok=True)
```

```
logger.info("All databases initialized successfully")
```

7. 資料備份與還原

7.1 SQLite 備份

```
#!/bin/bash
```

```
# backup-sqlite.sh
```

```
DATE=$(date +%Y%m%d_%H%M%S)
```

```
BACKUP_DIR="/backup/sqlite"
```

```
# 備份 skill_metadata.db
```

```
sqlite3 /app/data/skill_metadata.db ".backup '${BACKUP_DIR}/skill_metadata_${D/
```

```
# 備份 docai.db
```

```
sqlite3 /app/data/docai.db ".backup '${BACKUP_DIR}/docai_${DATE}.db'"
```

```
echo "SQLite backup completed: ${DATE}"
```

7.2 MongoDB 備份

```
#!/bin/bash
```

```
# backup-mongodb.sh
```

```
DATE=$(date +%Y%m%d_%H%M%S)
```

```

BACKUP_DIR="/backup/mongodb"

# 使用 mongodump
docker exec docai-mongodb mongodump \
    --db docai \
    --out /data/backup/${DATE}

# 複製到主機
docker cp docai-mongodb:/data/backup/${DATE} ${BACKUP_DIR}/

echo "MongoDB backup completed: ${DATE}"

```

7.3 FAISS 備份

```

#!/bin/bash
# backup-faiss.sh

DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR="/backup/faiss"

# 壓縮整個 FAISS 索引目錄
tar -czvf ${BACKUP_DIR}/faiss_indices_${DATE}.tar.gz \
    /app/data/faiss_indices/

echo "FAISS backup completed: ${DATE}"

```

附錄 A: 環境變數參考

```

# =====
# SQLite 設定
# =====
SQLITE_DB_PATH=/app/data/skill_metadata.db
DATABASE_URL=sqlite:///app/data/skill_metadata.db

# =====
# MongoDB 設定
# =====
MONGODB_URI=mongodb://mongodb:27017          # 容器內
# MONGODB_URI=mongodb://localhost:27017      # 本機開發
MONGODB_DATABASE=docai
MONGODB_COLLECTION=chat_history

# =====
# Redis 設定
# =====
REDIS_URL=redis://redis:6379/0                # 容器內
# REDIS_URL=redis://localhost:6379/0          # 本機開發
REDIS_HOST=redis

```

```
REDIS_PORT=6379
REDIS_DB=0
CACHE_TTL=3600
```

```
# =====
# FAISS / Vector Store 設定
# =====
VECTOR_STORE_TYPE=faiss
VECTOR_STORE_PATH=/app/data/faiss_indices
EMBEDDING_MODEL=BAAI/bge-m3
EMBEDDING_DIMENSION=1024
```

附錄 B: 資料庫版本遷移

```
# migrations/001_initial_schema.py

async def upgrade():
    """初始資料庫結構"""
    # SQLite tables 建立...
    pass

async def downgrade():
    """還原初始結構"""
    # DROP TABLES...
    pass

# migrations/002_add_head_id.py

async def upgrade():
    """新增 head_id 欄位到 skill_metadata"""
    await db.execute("""
        ALTER TABLE skill_metadata
        ADD COLUMN head_id TEXT REFERENCES skill_heads(head_id)
    """)

async def downgrade():
    """移除 head_id 欄位"""
    # SQLite 不支援 DROP COLUMN，需要重建表格
    pass
```

文檔結束

Generated by SuperClaude Framework for DocAI Containerization Project